

Assignment 1 - PS1: Smart Waste Collection Agent

Design Document

1. Problem Statement

Bengaluru's Smart City waste management program needs a robot to travel between collection centers and processing units as cheaply as possible. The road network is represented as a weighted undirected graph: nodes are locations and each edge carries a cost reflecting fuel or travel time. Given a source and a destination, the task is to find the least-cost path between them. This is well-suited to informed search because the state space is discrete, costs are non-negative, and a meaningful heuristic can be constructed from graph structure. IDA* is chosen because it is both optimal and memory-efficient, which is important when the robot has limited on-board resources.

2. PEAS Analysis

Performance: Find the minimum-cost path from source to destination. Lower total travel cost means better performance.

Environment: A static, weighted undirected graph of Bengaluru waste management locations. Edge weights are positive integers representing travel cost.

Actuators: The robot moves along an edge from its current node to an adjacent node.

Sensors: The robot knows its current location, the edges leaving it (with costs), and the identity of the goal node.

The agent is goal-based and operates in a fully observable, deterministic, static, discrete environment.

3. Heuristic Function

The central design question for IDA* is how to estimate the remaining cost from a node to the goal without actually solving the problem. The estimate must be a lower bound so that IDA* can safely prune paths and still guarantee optimality.

Our heuristic uses two pieces of graph structure: the minimum number of hops from a node to the goal, and the smallest edge weight in the graph. The heuristic is computed as:

$$h(n) = \text{min_hops}(n, \text{goal}) \times \text{min_edge_weight}$$

This is admissible because any path from n to the goal must traverse at least $\text{min_hops}(n, \text{goal})$ edges, and each edge costs at least min_edge_weight . It is also consistent because moving one hop closer can reduce h by at most min_edge_weight , which is never greater than the actual edge cost. These properties ensure IDA* finds the optimal path.

4. Cost Function and Search Mechanics

IDA* evaluates each node using $f(n) = g(n) + h(n)$, where $g(n)$ is the actual cost from the source to node n and $h(n)$ is the heuristic estimate of the remaining cost. The algorithm starts with a threshold equal to $h(\text{source})$ and performs a depth-first search. Any branch where $f(n)$ exceeds the threshold is pruned. If the goal is not found, the threshold is raised to the smallest pruned f -value and the search repeats.

5. Algorithm Choice — Why IDA*?

A* is optimal with an admissible heuristic but stores the frontier in memory, which can grow large. IDA* avoids this by only remembering the current path, using linear memory $O(d)$ where d is the search depth.

Dijkstra is also optimal for non-negative weights, but it requires $O(V)$ memory for the visited set and priority queue. We chose IDA* because the robot is resource-constrained and the lower memory footprint is a sensible trade-off for the moderate node re-expansion that occurs across iterations.

6. Data Structures Used

PathStack: A bounded stack maintaining the current DFS path. Its capacity is set to the number of nodes in the graph. `push()` reports stack overflow if the stack is full and `pop()` reports stack underflow if the stack is empty. A companion hash set provides $O(1)$ membership checks to avoid cycles.

Adjacency List: The graph is stored as a dictionary mapping each node to a list of (neighbor, weight) tuples. Since roads are bidirectional, each edge is added in both directions.

BFS Queue (deque): Used once during heuristic precomputation to compute hop distances from the goal to all reachable nodes.

7. Implementation Overview

Input is read from a plain text file. The parser now requires explicit edge lines of the form `node1 node2 weight` and rejects compact tokens such as `AB3`. The command-line script accepts both an exact filename or a regex pattern that matches exactly one file. The graph is built as an undirected adjacency list, and heuristic values are precomputed in one BFS pass from the goal. IDA* is implemented as recursive DFS using PathStack for path tracking and cycle detection. When $f(n)$ exceeds the current threshold, the branch is pruned and the exceeded value contributes to the next threshold. The visited sequence is tracked per iteration and includes pruned nodes.

8. Error Handling

The code handles missing or unreadable input files, empty files, malformed edge lines, non-positive weights, blank source or destination values, source or destination not present in the graph, disconnected graphs, `source == destination`, stack overflow, and stack underflow.

9. Alternate Approach — Dijkstra's Algorithm

Dijkstra's algorithm is a natural alternative. It processes nodes in order of accumulated cost and is optimal for non-negative weights. Compared to IDA*, Dijkstra uses more memory ($O(V)$) because it stores the frontier in a priority queue, but it does not re-expand nodes once they are settled. For this problem size either algorithm is viable, but IDA* better demonstrates informed search while minimizing memory usage.

10. Test Cases and Sample Output

Case 1 — MG_Road to Yelahanka:

Iteration 1 evaluated: MG_Road -> Electronic_City -> Whitefield -> Yelahanka -> Jayanagar -> Koramangala

Iteration 2 evaluated: MG_Road -> Electronic_City -> Whitefield -> Yelahanka

Optimal Path: MG_Road -> Electronic_City -> Whitefield -> Yelahanka

Cost: 8

Nodes Explored: 10 (includes pruned nodes)

Case 2 — A to E:

Iteration 1 evaluated: A -> B -> C

Iteration 2 evaluated: A -> B -> C -> D -> E

Iteration 3 evaluated: A -> B -> C -> D -> B -> E -> E

Iteration 4 evaluated: A -> B -> D -> C -> D -> B -> E

Optimal Path: A -> C -> D -> E

Cost: 5

Nodes Explored: 22 (includes pruned nodes)

11. How to Run

Open `ida_star_waste_collection.ipynb` in Jupyter or run the module directly from the command line with input and output file paths. The updated script also supports a regex input pattern that matches exactly one file name.

Command example:

```
python3 ida_star_waste_collection.py inputPSXX.txt outputPSXX.txt
```